Software Engineering Foundations

From source code to reliable systems: a practical guide to version control, functions, testing, and API contracts.

Course goal
Learn how small engineering decisions combine into software that is understandable, testable, and safe to change.

WHAT YOU WILL PRACTICE

You will trace a change from an isolated Git branch through a small function, an automated test, and a stable API response. Each section introduces one boundary that helps teams reason about a larger system.

01 Source control makes change visible

A version-controlled project records not only the current files but also the history of decisions that produced them. A branch gives an engineer a safe place to explore a change without disrupting the shared main branch. Small, focused commits create a readable trail from a bug report to its fix.

- Create a branch for one concern instead of mixing unrelated edits.
- Write commit messages that explain intent, not just file changes.
- Review the diff before merging so accidental changes remain visible.

Rule of thumb
If a commit is difficult to describe in one sentence, it probably contains more than one logical change.

02 Functions create useful boundaries

A function should have a clear responsibility, explicit inputs, and a result that another part of the program can trust. Keeping calculation separate from input and output makes the calculation easier to test. Pure functions are especially useful because the same inputs always produce the same result and they do not hide state changes.

GOOD BOUNDARY CHECKLIST

- Does the function name describe one observable outcome?
- Are invalid inputs handled close to the boundary?
- Can the core logic run without a browser, network, or database?

Errors are part of an interface. A function that can fail should communicate that failure in a predictable way. Returning a clear validation error is often better than allowing an invalid value to travel deeper into the application.

> **Example**
> `parseScore(input) -> score` should either return a valid number from 0 to 100 or raise a specific invalid-input error. Callers can then choose how to present the problem.

03 Tests turn expectations into feedback

An automated test describes a behavior the team wants to preserve. The arrange, act, assert pattern makes the scenario readable: prepare input, execute the operation, and compare the result with the expected outcome. A failing test is valuable evidence because it narrows the search to a specific contract.

PRACTICAL TESTING SEQUENCE

1. Reproduce the smallest failing case.
2. Fix the cause rather than weakening the assertion.
3. Re-run related tests to catch regressions.

04 APIs communicate contracts

An API response is more than data: it is a promise about field names, types, status codes, and failure behavior. Client code should validate what it receives before trusting that shape. A schema check turns a vague integration failure into a useful, local error message.

A RELIABLE API CONTRACT SPECIFIES

- The request fields that are required and their accepted types.
- The successful response shape and the meaning of each field.
- How authentication, rate limits, and server failures are reported.

Status codes help different layers react appropriately. For example, a 401 response usually means credentials must be corrected, while a 429 response means the client should slow down and retry later. Treating every failure as a generic network error hides information the user needs.

Review before shipping

Check the change against four questions: Is the intent visible in the diff? Does the core function have a clear contract? Do tests cover the expected and failing paths? Can the consumer identify and recover from an API error? These checks are small, repeatable habits that make software easier to evolve.

> **Key takeaway**
> Good engineering reduces uncertainty at every boundary: between versions, between functions, between tests and behavior, and between services.

Terms to remember

Branch: an independent line of development. **Pure function:** a function whose result depends only on its inputs. **Regression test:** a test that protects an existing behavior after a change. **Contract:** the agreed inputs, outputs, and failure modes of an interface.